

HELLO!

I am Sayeed Unnisa

I am here because I love to give presentations
on **Java Full Stack**

Advance Java JDBC Technology (API)

JDBC [Java Data Base Connectivity]:

API: API is a set of classes & Interfaces used to perform any task.

API→Application Programming Interface .

JDBC API :

JDBC stands for java database connectivity. It is an API (Application Programming Interface) given by Sun Microsystems to perform “CURD” operations with Database using Java.

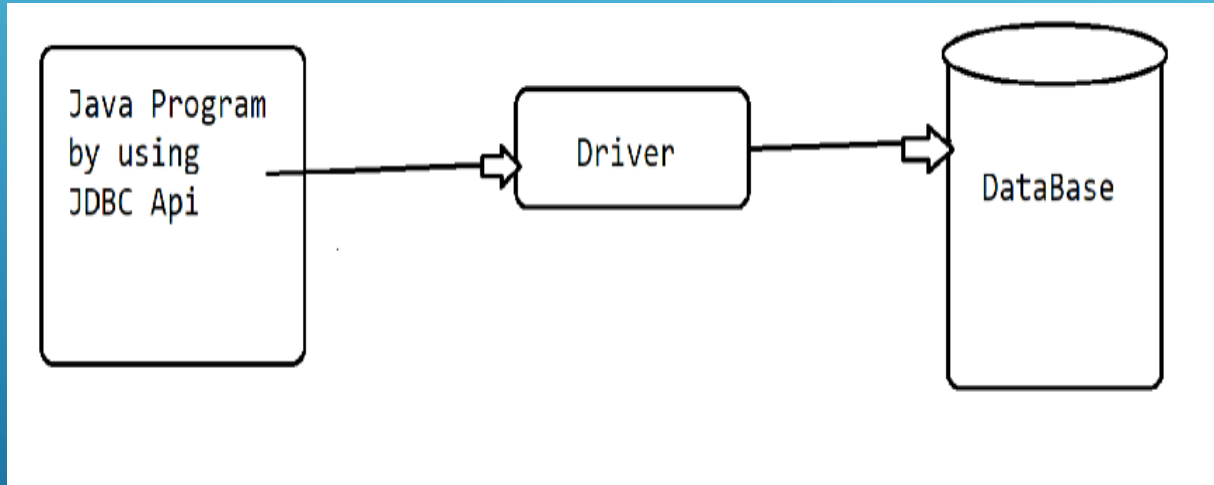
C=Create (Create Database/Tables/Rows...)

U=Update (modify tables/rows)

R=Read (Selecting data from Database)

D=Delete (remove rows/tables/DBs)

JDBC Architecture:



Driver:

It is a pre-defined class (in simple words java software) which creates a communication link between JDBC & Database.

- Without Driver no JDBC concept works.
- For Every Database Driver Software (class) is required.
- DriverManager is a class which takes care of Driver.

Example Driver Names:

Oracle Driver = For oracle DataBase

MySQLDriver = For MySQL DataBase

- JDBC API means classes and interfaces defined in two packages. Those are:
 1. Java.Sql
 2. Javax.Sql (Extended)
- Few classes and interfaces in above packages are given as connection, Statement, PreparedStatement, ResultSet, DatabaseMetadata, RowSet etc.
- To work with JDBC, our output should be converted to SQL which can be understandable by Database.

Writing JDBC Application:

Here Programmer has to follow below steps, to do any operation (Select or non-select) with Database.

Those are:

1. **Load Driver class:**-It will make a link between java Application and Database software. It is a class in JDBC which implements Driver interface.
2. **Creating connection:**-To create connection we should provide location of Database (URL) and username with password of Database for Access Permission to java.
3. **Create Statement:**-It indicates one SQL query which should be given to DB. For one operation one SQL= one statement.
4. **Execute Statement:**-Give above SQL to DB and execute it.
5. **Get result:**-Final result of above SQL Query execution is given as either ResultSet obj (or) int/Boolean.

6. **Print result:**-Print above result using Sysout statement.

7. **Close connection:**-Finally close connection between Java Application & Driver (class), url, username, password.

Connecting with MYSQL database:

Download :

MySQL SOFTWARE

MYSQL JAR

MYSQL JDBC Properties:

Driver Class

URL

IP ADDRESS

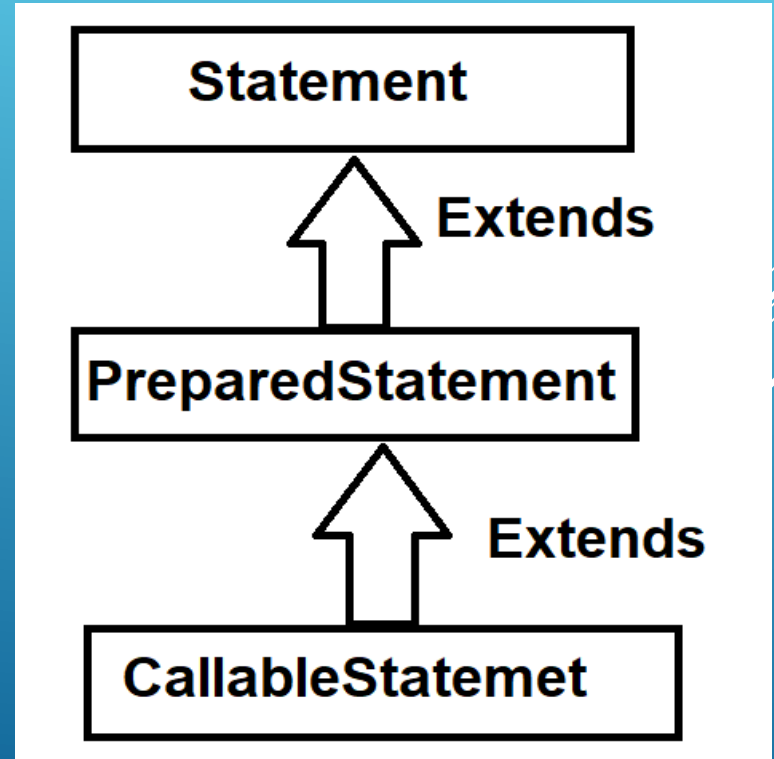
<PORT_No.>

/<dbname>

Ex: jdbc:mysql://localhost:3306/test

Types of statement interfaces in JDBC:

1. Statement(I)
2. PreparedStatement(I)
3. CallableStatement(I)



To Execute The SQL Commands, there are three methods given by Statement Interface.

1. `executeUpdate()`
2. `executeQuery()`
3. `execute()`

Fetching Data from Database using JDBC:

- Write “select SQL” query to get from Database to java Application.
- Here Data will be stored in ResultSet object format .
- `java.sql.ResultSet()` is a interface it's implementation class object is created by using method : `executeQuery(String sql)`.
- ResultSet memory is created using column index, column label, column data type and Data in Rows format. Index number start from 1 (one).
- ResultSet contains methods like:
- `next()`: Boolean => this method is used to check next row exist or not in ResultSet.
 `get <DataType> (int columnIndex) (or) get<DataType> (String columnLable)`
- These methods are used to read data from ResultSet.
- Index number may change based on query select column order. But column labels will never get changed.

PreparedStatement:

PreparedStatement is sub interface of Statement .

there are mainly 2 problems with Statement object

1. Statement object can not store compiled code of sql command.

so every time command is compiled it decreases the performance

**2. Statement object can only transfer character data from java to db
but not binary data**

The above two problems can be avoided with PreparedStatement while creating a PreparedStatement object, sql command will be sent to DB and it is compiled in DB then returns the compiled code to PreparedStatement object

we can execute the already compiled code for any no. of times

while creating PreparedStatement object, only command syntax required

without values so, values are skipped by putting index parameters(?)

```
PreparedStatement pstmt=con.prepareStatement("insert into  
student values(?,?,?)");
```

to execute compiled code for one or more times, we need to set the values for index parameters by calling setXxx() methods

CallableStatement :

- It is an interface, which extends PreparedStatement.
- It has all the benefits of PreparedStatement and with CallableStatement we can call the procedure and function of DB.
- In real-time projects, at DB side, a procedure or function will be created by DB administrator.
- Java Developer can only call a procedure or function by using CallableStatement.
- CallableStatement is used only for calling a procedure or a function but not for creating them.
- In DB, a procedure or function is created for calculations by using data stored in DB.
- After performing calculation, If there is a single value to return then a function is created. If multiple values are there to return then procedure is created.

- CallableStatement object can be created in 2 syntax,

Syntax:

1. `CallableStatement cstmt = con.prepareCall("Sql Command");`
2. `CallableStatement cstmt= con.prepareCall("{call Procedure_name() / function_name()}");`

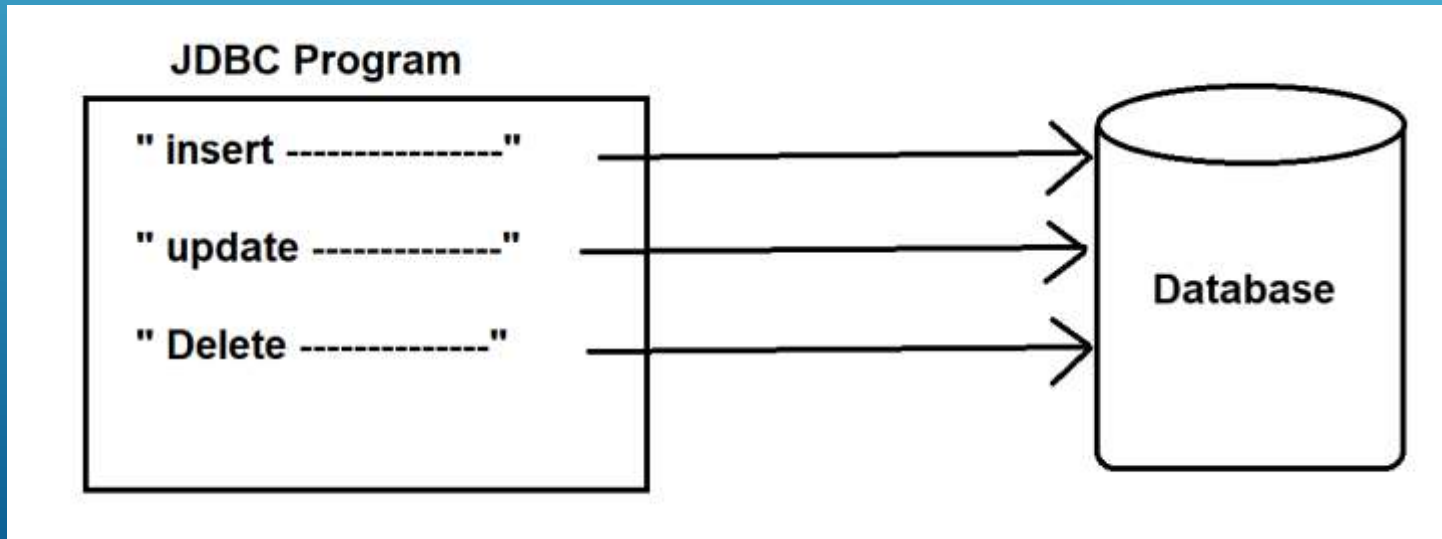
- CallableStatement object also store compiled code, like PreparedStatement.

Ex:

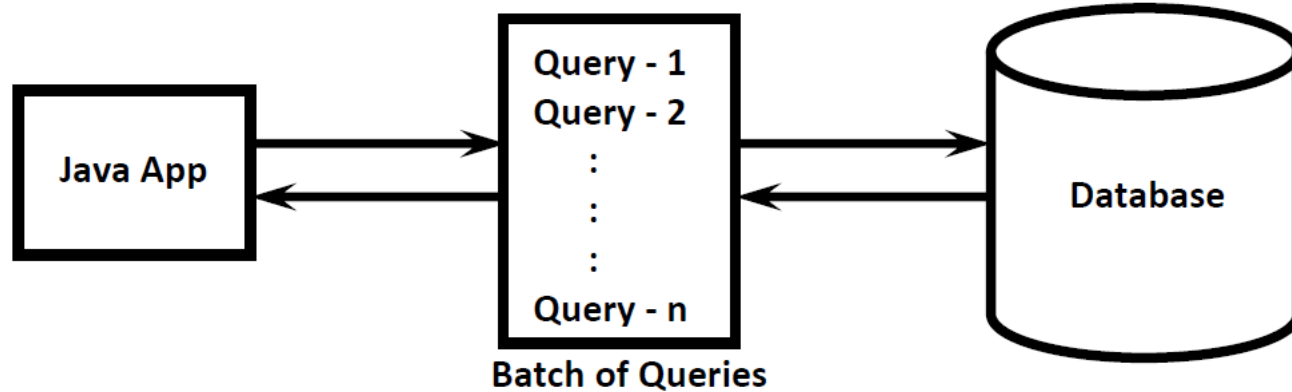
```
CallableStatement cstmt= con.prepareCall("update  
Employee set salary=? Where id=?");
```

Batch Processing:

- ❑ If we write multiple commands in a JDBC program then one by one command will be send to DB for execution.
- ❑ If one by one command is send to DB then a program performance will be decreased, because number of trips between program and DB are increased.



- Batch processing concept is given to improve the performance.
- In Batch processing, multiple commands will be send to DB in a single trip. So, that number of trips will be reduced between JDBC program and DB. So, performance is going to be improved.



Batch Processing with PreparedStatement :

- Batch Processing can be done by PreparedStatement.
- The difference between PreparedStatement and Statement batch processing is, in Statement object we can add different commands to the batch but in PreparedStatement, we can add same sql command for batch with different values.

Transaction Management in JDBC:

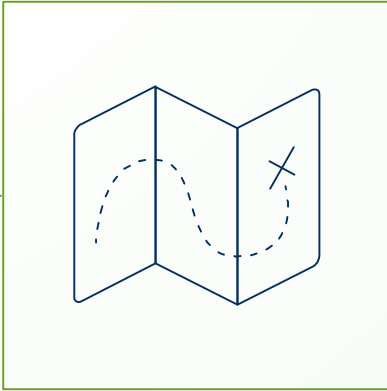
- Process of combining all related operations into a single unit and executing on the rule "either all or none", is called transaction management.
- Hence transaction is a single unit of work and it will work on the rule "either all or none".
- Case-1: Funds Transfer
 - 1. debit funds from sender's account
 - 2. credit funds into receiver's account
 - All operations should be performed as a single unit only. If debit from sender's account completed and credit into receiver's account fails then there may be a chance of data inconsistency problems.

❖ **Case-2: Movie Ticket Reservation**

- 1. Verify the status
- 2. Reserve the tickets
- 3. Payment
- 4. issue tickets.
- All operations should be performed as a single unit only. If some operations success and some operations fails then there may be data inconsistency problems.

Process of Transaction Management in JDBC:

- 1. Disable auto commit mode of JDBC
- By default auto commit mode is enabled. i.e after executing every sql query, the changes will be committed automatically in the database.
- We can disable auto commit mode as follows
- `setAutoCommit(false);`
- 2. If all operations completed then we can commit the transaction by using the following method.
- `commit();`
- 3. If any sql query fails then we have to rollback operations which are already completed by using `rollback()` method.
- `rollback();`



THANKS!

Any questions?

You can find me at
sayeedunnisa.js@gmail.com